Title: Deadlock Detection, Avoidance, and Recovery Using Banker's Algorithm

1. AIM

To study the concept of deadlocks in operating systems and implement the Banker's Algorithm in C to check whether the system is in a safe state or an unsafe state.

2. THEORY

2.1 Introduction to Deadlock

A deadlock is a situation in an operating system in which two or more processes are blocked because each process is waiting for a resource held by another process.

In this situation, none of the processes can continue execution because the required resources are unavailable.

Example:

- Process P1 holds Resource R1 and waits for Resource R2.
- Process P2 holds Resource R2 and waits for Resource R1.
- Both processes wait indefinitely.

2.2 Example of Deadlock

```
None
        Process P1
            |
            | Holds R1
            |
            v
        Resource R1
            |
            | Requested by P2
            |
            v
        Process P2
            |
```

```
        | Holds R2
        |
        v
    Resource R2
        |
        | Requested by P1
        |
        +----------> Deadlock
```

2.3 Necessary Conditions for Deadlock

A deadlock can occur when all four of the following conditions exist simultaneously.

1. Mutual Exclusion

At least one resource must be held in a non-shareable mode. Only one process can use that resource at a time.

2. Hold and Wait

A process holds one or more resources while waiting for additional resources.

3. No Preemption

Resources cannot be forcibly taken away from a process. They must be released voluntarily.

4. Circular Wait

A circular chain of processes exists in which each process waits for a resource held by the next process.

2.4 Deadlock Handling Methods

Operating systems use different approaches to handle deadlocks.

Method	Description
Deadlock Prevention	Prevents at least one necessary deadlock condition

Deadlock Avoidance	Allocates resources only when the system remains safe
Deadlock Detection	Checks whether a deadlock has occurred
Deadlock Recovery	Takes action to resolve an existing deadlock

2.5 Deadlock Avoidance

Deadlock avoidance is a technique in which the operating system checks resource allocation requests before granting them.

A resource request is granted only if the resulting system state is safe.

A safe state means that there is at least one sequence in which all processes can complete their execution.

2.6 Banker's Algorithm

Banker's Algorithm is a deadlock avoidance algorithm developed by Edsger W. Dijkstra.

It checks whether allocating resources to a process will keep the system in a safe state.

The algorithm uses the following data structures:

Data Structure	**Meaning**
Available	Number of available instances of each resource
Max	Maximum resource requirement of each process
Allocation	Resources currently allocated to each process

Need	Remaining resources required by each process

2.7 Need Matrix Calculation

The Need matrix is calculated using:

$$Need = Max - Allocation$$

For example:

Process	Max	Allocation	Need
P0	7, 5, 3	0, 1, 0	7, 4, 3
P1	3, 2, 2	2, 0, 0	1, 2, 2
P2	9, 0, 2	3, 0, 2	6, 0, 0

2.8 Safe State

A system is in a safe state if there is at least one safe sequence of process execution.

A safe sequence is an order in which every process can obtain the resources it still needs, complete execution and release its allocated resources.

The algorithm checks:

$$Need[i] \leq Available$$

If the condition is satisfied, the process can complete using the currently available resources.

After completion:

$$Available = Available + Allocation[i]$$

2.9 Unsafe State

An unsafe state is a state in which the system cannot guarantee that all processes can finish.

An unsafe state does not necessarily mean that deadlock has already occurred. It means that the system has lost the guarantee of a safe execution sequence.

2.10 Working of Banker's Algorithm

1. Initialize the Available, Max and Allocation matrices.
2. Calculate the Need matrix.
3. Find a process whose Need is less than or equal to Available.
4. Assume that process completes execution.
5. Add its allocated resources back to Available.
6. Repeat until all processes finish or no eligible process can be found.
7. If all processes finish, the system is safe.
8. Otherwise, the system is unsafe.

2.11 Advantages of Banker's Algorithm

1. Helps avoid deadlock situations.
2. Checks system safety before resource allocation.
3. Provides a safe execution sequence.
4. Helps manage multiple resource types.
5. Improves resource allocation decisions.

2.12 Limitations of Banker's Algorithm

1. Requires prior knowledge of maximum resource requirements.
2. Resource requirements should remain predictable.
3. The algorithm involves additional computation.
4. It may not be suitable for highly dynamic systems.
5. It can reduce resource utilization because some requests may be delayed.

3. CODE

Program: Implementation of Banker's Algorithm in C

Filename: `bankers.c`

The following program calculates the Need matrix and checks whether the system is in a safe state. If the system is safe, it displays the safe sequence.

None

```
#include <stdio.h>

int main()
{
    int n, m;

    int allocation[10][10];
    int max[10][10];
    int need[10][10];

    int available[10];
    int finish[10] = {0};

    int safeSequence[10];

    int i, j, k;
    int count = 0;
    int found;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    // Input Allocation Matrix
    printf("\nEnter Allocation Matrix:\n");

    for (i = 0; i < n; i++)
    {
        for (j = 0; j < m; j++)
        {
            scanf("%d", &allocation[i][j]);
        }
    }
```

```
// Input Maximum Matrix
printf("\nEnter Maximum Matrix:\n");

for (i = 0; i < n; i++)
{
    for (j = 0; j < m; j++)
    {
        scanf("%d", &max[i][j]);
    }
}

// Input Available Resources
printf("\nEnter Available Resources:\n");

for (i = 0; i < m; i++)
{
    scanf("%d", &available[i]);
}

// Calculate Need Matrix
printf("\nNeed Matrix:\n");

for (i = 0; i < n; i++)
{
    for (j = 0; j < m; j++)
    {
        need[i][j] = max[i][j] - allocation[i][j];

        printf("%d ", need[i][j]);
    }

    printf("\n");
}

// Banker's Algorithm
while (count < n)
```

```
{
    found = 0;

    for (i = 0; i < n; i++)
    {
        if (finish[i] == 0)
        {
            int canExecute = 1;

            for (j = 0; j < m; j++)
            {
                if (need[i][j] > available[j])
                {
                    canExecute = 0;
                    break;
                }
            }

            if (canExecute)
            {
                for (k = 0; k < m; k++)
                {
                    available[k] += allocation[i][k];
                }

                safeSequence[count] = i;

                count++;

                finish[i] = 1;

                found = 1;
            }
        }
    }
```

```
        if (found == 0)
        {
            break;
        }
    }

    // Display Result
    if (count == n)
    {
        printf("\nSystem is in SAFE STATE.\n");

        printf("Safe Sequence: ");

        for (i = 0; i < n; i++)
        {
            printf("P%d", safeSequence[i]);

            if (i != n - 1)
            {
                printf(" -> ");
            }
        }

        printf("\n");
    }
    else
    {
        printf("\nSystem is in UNSAFE STATE.\n");

        printf("No safe sequence exists.\n");
    }

    return 0;
}
```

Explanation of the Program

1. Input Processes and Resources

```
None
scanf("%d", &n);
scanf("%d", &m);
```

Accepts the number of processes and resource types.

2. Allocation Matrix

```
None
int allocation[10][10];
```

Stores the resources currently allocated to each process.

3. Maximum Matrix

```
None
int max[10][10];
```

Stores the maximum resource requirement of each process.

4. Need Matrix

```
None
need[i][j] = max[i][j] - allocation[i][j];
```

Calculates the remaining resources required by each process.

5. Available Resources

```
None
int available[10];
```

Stores the currently available resource instances.

6. Safety Check

```
None
if (need[i][j] > available[j])
```

Checks whether the process can complete with the currently available resources.

7. Resource Release

```
None
available[k] += allocation[i][k];
```

When a process completes, its allocated resources are returned to the available resource pool.

8. Safe Sequence

```
None
safeSequence[count] = i;
```

Stores the order in which processes can safely execute.

4. EXECUTION INSTRUCTIONS

Requirements

- Operating System: Ubuntu Linux / Kali Linux.
- Programming Language: C.
- Compiler: GCC.
- Terminal: Linux Terminal.

Step 1: Open Terminal

Press:

```
None
Ctrl + Alt + T
```

Step 2: Check GCC Compiler

```
None
gcc --version
```

If GCC is not installed:

```
None
sudo apt update
sudo apt install build-essential
```

Step 3: Create Practical Directory

```
None
mkdir Bankers_Algorithm
cd Bankers_Algorithm
```

Step 4: Create C Program

```
None
nano bankers.c
```

Copy and paste the above program.

Save the file:

- Press `Ctrl + O`.
- Press Enter.
- Press `Ctrl + X`.

Step 5: Compile the Program

```
None
gcc bankers.c -o bankers
```

Step 6: Execute the Program

```
None
./bankers
```

Step 7: Enter Sample Input

Use the following sample input.

Number of processes and resources

```
None
5
3
```

Allocation Matrix

```
None
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
```

Maximum Matrix

```
None
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
```

Available Resources

```
None
3 3 2
```

Step 8: Expected Output

```
None
Need Matrix:
```

```
7 4 3
1 2 2
6 0 0
0 1 1
4 3 1

System is in SAFE STATE.

Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2
```

Expected result: Safe state

Note: The program may find a different valid safe sequence depending on the order in which eligible processes are checked.

5. RESULT

The Banker's Algorithm was successfully implemented using C programming in the Linux environment.

The following operations were performed:

1. Accepted the number of processes and resources.
2. Entered the Allocation and Maximum matrices.
3. Calculated the Need matrix.
4. Checked available resources for each process.
5. Identified processes that could safely execute.
6. Calculated the safe sequence.
7. Determined whether the system was in a safe or unsafe state.

The sample input produced a safe execution sequence:

P1 → P3 → P4 → P0 → P2

Therefore, the system was found to be in a safe state.

6. CONCLUSION

In this practical, the concept of deadlock detection, avoidance and recovery was studied. The Banker's Algorithm was implemented using C programming to determine whether a system is in a safe or unsafe state. The Allocation, Maximum, Available and Need matrices were used to

analyze resource allocation. The algorithm successfully identified a safe sequence of process execution and demonstrated how deadlock avoidance can be achieved through proper resource management.

Practical 8 Completed

All 8 Operating Systems practicals are now covered.